

Autómatas de protocolos

Redes de Comunicaciones - Curso 2023/24

Introducción

Durante esta sesión vamos a analizar la temporización o secuencia correcta del intercambio de mensajes de un protocolo, que normalmente suele especificarse mediante modelos computacionales como los autómatas de estados finitos.

Autómatas finitos

A la hora de diseñar un protocolo de comunicación, además del formato exacto de los mensajes, también necesitamos especificar cuál es la secuencia correcta de intercambio de los mismos, puesto que no todas las combinaciones de envío de mensajes serán posibles ni tendrán sentido. La forma habitual de concretar las secuencias válidas es mediante modelos computacionales como los autómatas (o máquinas de estados) finitos. Dado que dichos modelos son vistos en detalle en otras materias de la titulación, así como en las clases teóricas de la asignatura de Redes de Comunicaciones, en esta sesión nos limitaremos a elaborar autómatas concretos basados en la práctica a desarrollar en la asignatura.

A modo de recordatorio, indicaremos que un autómata finito puede considerarse como una quintupla $(Q, \Sigma, \delta, q_s, F)$ en la que:

- Q es el conjunto de estados.
- Σ es el alfabeto de entrada o secuencia de eventos.
- δ es el conjunto de transiciones entre los estados.
- q_s es el estado inicial del autómata.
- F es el conjunto de estados finales del autómata.

Se estudiarán diferentes autómatas que irán aumentando de complejidad conforme se introduzcan nuevos elementos y condiciones.

En esta asignatura, la especificación de los autómatas se realizará mediante el grafo de transiciones, que permite visualizar la quintupla por completo. Además, Σ se definirá en detalle cuando se definan los mensajes y su formato en la sección correspondiente de la documentación que los alumnos deben entregar con el protocolo diseñado.

Existen varias formas de plasmar un protocolo en forma de un autómata finito. Nosotros tomaremos una estrategia para el diseño del protocolo basada en:

- *eventos de interés* (p. ej., se envía un mensaje). El evento de envío de un cierto mensaje se denotará en el autómata mediante **snd(message)**, mientras que la recepción se denota con **rcv(message)**. Entre paréntesis se indicará el tipo de mensaje enviado o recibido (operación u *opcode*).
- *estados que requieren un procesamiento o una acción concreta* (p. ej., se agota el tiempo establecido en un temporizador sin haber recibido respuesta).

Autómatas para un protocolo cliente-servidor basado en canal confiable

En primer lugar, vamos a considerar el caso de un protocolo dado cuando el canal subyacente es confiable (e.g., TCP). Siguiendo la notación que se emplea en las clases teóricas para el análisis de los protocolos confiables, debemos considerar una representación del protocolo basada en dos autómatas; un autómata para el cliente y otro para el servidor. Ciertamente, ambos autómatas tienen estados comunes en la mayoría de los casos dado que deben mantener la secuencia de mensajes intercambiados. Por ejemplo, vamos a comenzar por dos autómatas que reflejan el comportamiento de un cliente cuando está solicitando descargar un fichero (p.ej., comando *downloadfrom*) y el servidor le responde. Dichos autómatas serían:

Autómata para el rol cliente (receptor de ficheros)

Autómata para el rol servidor de ficheros



Como podemos comprobar, la única diferencia entre ellos, en este caso, está en quién es el encargado de enviar o recibir los mensajes. Cabe destacar que es posible recibir respuestas distintas a un cierto mensaje enviado (p.ej., recibir un mensaje *FileNotFound* o bien recibir *File* en respuesta al envío de un mensaje *Download*). Estas respuestas, además, podrían dejar al programa en un estado distinto y, por tanto, con capacidad de ejecutar distintos comandos a posteriori. En estos ejemplos no se ha establecido estado inicial ni final(es) con la intención de que estas transiciones emularan una pequeña parte de un autómata más grande. Siendo estrictos, no son autómatas bien formalizados. Por otra parte, hay que tener en cuenta que no siempre el autómata del servidor se comportará como un reflejo del autómata cliente. Algunas peticiones pueden requerir que el servidor realice ciertas acciones que involucren eventos en la red que deben aparecer en su propio autómata con la secuenciación correcta.

Autómatas para un protocolo cliente-servidor con posibilidad de error y confirmaciones

En algunos casos, cuando el canal subyacente no es confiable, tenemos que tener en cuenta la posibilidad de que se pierdan algunos mensajes. Vamos a ver cómo se llevaría a cabo la especificación de los autómatas para esos casos.

Como es posible imaginar, la complejidad de los autómatas aumenta cuantas más condiciones de error debemos considerar. En este apartado vamos a ver el diseño de un autómata en el que se contempla que tanto las pérdidas como los errores tienen la misma consecuencia: el canal subyacente descarta el mensaje.

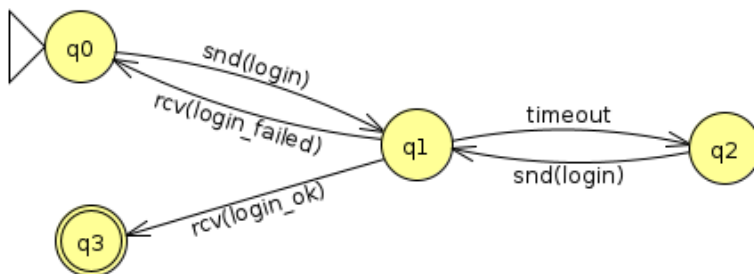
Vamos a utilizar como base el intercambio de mensajes entre el cliente de *NanoFiles* y el servidor de directorio que ya se empezó a trabajar en boletines anteriores.

Siguiendo la metodología de clase de teoría, un protocolo de parada y espera de este tipo que permita errores debe implementar dos mecanismos fundamentales:

- Envío de confirmaciones implícitas o explícitas.
- Uso de temporizadores (*timeouts*) de envío.

Este podría ser el autómata de un *peer* cliente para el envío del mensaje de inicio de sesión al servidor de directorio:

Autómata del cliente de directorio



Como se puede apreciar, este autómata tiene varias características:

- El estado inicial y final son distintos. Esto impide que tras un inicio de sesión exitoso se permita realizar de un nuevo ciclo de ejecución de una petición de *login*. Por tanto, este diseño impediría poder re-iniciar sesión para registrar un nuevo *nickname*. Esta decisión de diseño queda a discreción de los alumnos. En cualquier caso, es importante recalcar que en el diseño de los autómatas de las prácticas se debe buscar que cumplan con las siguientes restricciones:
 - Son autómatas mínimos.
 - Permiten seguir las iteraciones que admite el propio código que modela.
 - No aceptan ejecuciones que el código entregado no puede ejecutar.
- El mecanismo de *timeout* establecido es infinito. Esto es altamente indeseable. Se puede asumir que tras una serie de reintentos acotados en número, el servidor está caído y no va a responder. Al ser los autómatas unas máquinas de estados sin memoria, se recomienda a los alumnos definir un **evento de interés** llamado *timeout_n*, que se produciría con el evento de *timeout* n-ésimo con prioridad sobre el de *timeout* estándar.

Ejercicios a realizar

1. Basándote en el autómata del cliente de directorio mostrado anteriormente (disponible en el Aula Virtual como `ejemplo_no_confiable_NF_login.jff`), termina de añadir las transiciones necesarias para incluir un mecanismo de *timeouts* y retransmisión, que haga que el protocolo con el directorio sea confiable a pesar de operar sobre un canal no confiable como UDP.
2. Amplía el autómata del ejercicio anterior para dar cabida a las operaciones de cierre de sesión y consulta de la lista de usuarios logueados. Ten en cuenta que, por la especificación de la práctica, es obligatorio iniciar sesión en el directorio antes de poder realizar cualquier otra consulta al mismo. También es importante recordar que el directorio rechazará aquellos mensajes de clientes que no proporcionen una clave de sesión válida, salvo en el mensaje de inicio de sesión.
3. Diseñar el resto del autómata del cliente de directorio, teniendo en cuenta las mejoras planteadas que cada grupo de prácticas considere para su implementación. **NOTA: Considerar una mejora en esta fase del diseño no obliga a su implementación posterior, pero sí es requisito necesario.** Se deja a discreción de los alumnos decidir qué acciones es posible realizar según el estado del

autómata en el que nos encontremos en cada momento. Ejemplos de tales decisiones serían: ¿se puede pedir la lista de ficheros compartidos por otros *peers* en cualquier estado?, ¿se puede publicar la lista de ficheros que este peer comparte cualquier estado?, ¿se puede registrar un servidor en puerto cualquier estado?, entre otras. Las decisiones de diseño del protocolo adoptadas por cada grupo deberán reflejarse en el autómata.

4. Diseñar el resto del autómata del servidor de directorio. Se puede considerar que, si la *sessionKey* proporcionada por el cliente es válida, entonces el directorio siempre responderá a cualquier petición que se le haga (es decir, el directorio es "stateless", en tanto que no mantiene un registro de las interacciones anteriormente realizadas con un determinado cliente, salvo por el *login*).
5. Diseñar los dos autómatas, cliente y servidor, para el protocolo de transferencia de ficheros entre *peers*. Dichos autómatas regirán la comunicación entre un *peer* servidor de ficheros que ha ejecutado el comando `fgserve` o `bgserve`, y otro *peer* actuando como cliente, que ha ejecutado el comando `downloadfrom` o `download`. Al diseñar este protocolo, se han de tener en cuenta aspectos como que un fichero solicitado puede no encontrarse entre los disponibles (p.ej., el *hash* no concuerda o bien que se trata de una subcadena ambigua). También hay que tener en cuenta en el diseño de este protocolo las posibles mejoras que se piensen implementar (p.ej., descarga de fragmentos de un fichero en vez del contenido íntegro del mismo, necesario para la mejora de descarga de múltiples servidores).

Bibliografía

- JFLAP Tutorial: <http://www.jflap.org/tutorial/>.
- Apuntes de la asignatura de Teoría de Autómatas.